JEEVI ACADEMY

Data Science Course Series

NumPy — The Complete Guide

Numerical Python for Data Science

Beginner → Intermediate Level

Topics Covered	Course Details
Introduction & Setup	Level: Beginner to Intermediate
Array Basics & Operations	Language: Python 3.x
Broadcasting & UFunc	Includes: Code Examples
Linear Algebra & Advanced Ops	Academy: Jeevi Academy

1. What is NumPy?

NumPy (Numerical Python) is a free, open-source Python library that provides fast, multi-dimensional arrays and a wide collection of mathematical functions. It is the core building block for nearly all data science and machine learning work in Python.

In simple words: NumPy gives Python the ability to handle large collections of numbers quickly and efficiently — without slow loops.

NumPy at a Glance
Full Name : Numerical Python
Type : Open-source Python Library
Purpose : Fast numerical computing with multi-dimensional arrays
License : BSD (Free to use, modify, and distribute)
Website : numpy.org

2. History of NumPy

NumPy did not appear overnight. It evolved over many years from earlier scientific computing libraries.

Year	Milestone
1995	Jim Hugunin creates 'Numeric' — the first array-based numerical library for Python.
2001	A separate project 'Numarray' is launched with improvements over Numeric.
2005	Travis Oliphant unifies both projects into NumPy, combining the best of Numeric and Numarray.
2006	NumPy 1.0 is officially released to the public.
2010s	NumPy grows as Data Science and Machine Learning boom globally.
2020	NumPy publishes a landmark paper in Nature, recognising its central role in scientific computing.
Today	NumPy powers millions of applications worldwide in science, engineering, finance, and AI.

3. Who Created NumPy?

Travis Oliphant — The Creator

NumPy was primarily created by Travis Oliphant in 2005. He was a scientist and software developer who saw the need for a single, unified numerical computing library in Python.

Travis Oliphant — Profile	
Name	: Travis Oliphant
Profession	: Scientist, Software Developer, Entrepreneur
Education	: PhD in Biomedical Engineering (Mayo Clinic / Brigham Young University)
Key Work	: Created NumPy (2005), Co-created SciPy
Company	: Co-founded Anaconda (the popular Python distribution)
Legacy	: His work made Python the world's #1 Data Science language

> *Fun Fact: Travis Oliphant also created SciPy and co-founded Anaconda — the company behind the Anaconda distribution that most data science students use today!*

4. Why NumPy?

The Problem with Plain Python

Python lists are flexible but slow for large-scale numerical work. Consider multiplying every element in a list:

```
# Python way – slow, needs a loop
numbers = [1, 2, 3, 4, 5]
doubled = [x * 2 for x in numbers]    # Manual loop required

# NumPy way – fast, no loop needed
import numpy as np
numbers = np.array([1, 2, 3, 4, 5])
doubled = numbers * 2                  # Applies to all elements instantly
```

Python List vs NumPy Array

Feature	Comparison
Speed	NumPy: 10x–100x faster | Python List: Slow on large data
Memory	NumPy: Compact (fixed type) | Python List: More RAM (mixed types)
Operations	NumPy: Element-wise, no loop | Python List: Needs explicit loops
Data Type	NumPy: Single type (efficient) | Python List: Any mixed types
Math Built-in	NumPy: Hundreds of functions | Python List: Very limited
Dimensions	NumPy: N-dimensional arrays | Python List: Only 1D natively

5. Why Learn NumPy for Data Science?

In Data Science, ALL data — whether it is customer records, images, audio, or text — is ultimately represented as numbers in arrays. NumPy is the fundamental tool for managing this.

Major Libraries Built on NumPy
Pandas — Data analysis, DataFrames, time-series
Matplotlib — Charts, plots, and data visualisation
Scikit-learn — Machine learning algorithms
TensorFlow — Deep learning and neural networks
Keras — High-level deep learning API
SciPy — Advanced scientific computing
OpenCV — Image processing and computer vision

Key Reasons to Learn NumPy

- **Foundation:** Understand NumPy and all other DS libraries become much easier to learn
- **Speed:** Real datasets have millions of rows — NumPy handles them in milliseconds
- **Statistics:** Mean, median, std deviation, percentile — all built in
- **Matrix Math:** Machine learning uses matrix multiplication daily — NumPy makes it simple
- **Preprocessing:** Normalise, reshape, filter, and split data before feeding to ML models
- **Industry Standard:** Every data science job expects candidates to know NumPy well

SECTION A — INTRODUCTION

6. Install NumPy

Method 1 — pip (Standard Python)

```
pip install numpy

# Upgrade to latest version
pip install --upgrade numpy
```

Method 2 — Anaconda (Recommended for Students)

If you use Anaconda, NumPy is pre-installed. Open Anaconda Navigator, launch Jupyter Notebook, and start coding immediately.

```
# Install via conda (if not present)
conda install numpy
```

Verify Installation

```
import numpy as np
print(np.__version__)
# Example output: 1.26.4
```

7. Import NumPy

Always import NumPy using the alias 'np'. This is the universal convention used everywhere in Data Science.

```
import numpy as np          # Standard convention – always use 'np'

# Now you can use np.array(), np.zeros(), np.mean(), etc.
```

> *Why 'np'? It is shorter to type and is recognised instantly by every data scientist worldwide. Never name your file 'numpy.py' — it will conflict with the library.*

8. NumPy Arrays Basics

What is an ndarray?

The core object in NumPy is the ndarray (N-Dimensional Array). It is a grid of values, all of the same data type, and indexed by a tuple of non-negative integers.

Creating Arrays

From a Python List

```
# 1-D Array
a = np.array([10, 20, 30, 40, 50])
print(a)            # [10 20 30 40 50]
print(type(a))      # <class 'numpy.ndarray'>

# 2-D Array (matrix: rows x columns)
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b.shape)      # (2, 3)  -> 2 rows, 3 columns

# 3-D Array
c = np.array([[[1,2],[3,4]],[[5,6],[7,8]]])
print(c.shape)      # (2, 2, 2)
```

Built-in Array Creators

```
np.zeros((3, 4))         # 3x4 matrix of all 0s
np.ones((2, 3))          # 2x3 matrix of all 1s
np.full((3, 3), 7)       # 3x3 matrix filled with 7
np.eye(4)                # 4x4 identity matrix (diagonal = 1)
np.arange(0, 10, 2)      # [0, 2, 4, 6, 8]  (start, stop, step)
np.linspace(0, 1, 5)     # [0.0, 0.25, 0.5, 0.75, 1.0]
np.empty((2, 3))         # Uninitialised array (fast, garbage values)
```

Array Attributes

Every array has built-in properties to inspect its structure:

```
a = np.array([[1, 2, 3], [4, 5, 6]])

print(a.shape)     # (2, 3)   → rows, columns
print(a.ndim)      # 2        → number of dimensions
print(a.size)      # 6        → total elements
print(a.dtype)     # int64    → data type
print(a.itemsize)  # 8        → bytes per element
```

Data Types (dtype)

dtype	Description
`int32 / int64`	Integer numbers (32-bit or 64-bit)
`float32 / float64`	Decimal numbers (single or double precision)

`bool`	True / False values
`complex128`	Complex numbers (real + imaginary)
`str_`	Fixed-length Unicode strings

```
a = np.array([1, 2, 3], dtype=np.float64)   # Force float type
b = a.astype(np.int32)                        # Convert type
```

9. Indexing & Slicing

1-D Array Indexing

```
a = np.array([10, 20, 30, 40, 50])

print(a[0])        # 10    → first element
print(a[-1])       # 50    → last element
print(a[1:4])      # [20 30 40]
print(a[:3])       # [10 20 30]
print(a[::2])      # [10 30 50]  → every 2nd element
print(a[::-1])     # [50 40 30 20 10]  → reversed
```

2-D Array Indexing

```
b = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(b[0, 1])        # 2       → row 0, col 1
print(b[2, 2])        # 9       → row 2, col 2
print(b[1, :])        # [4 5 6] → entire row 1
print(b[:, 0])        # [1 4 7] → entire column 0
print(b[0:2, 1:3])    # [[2,3],[5,6]] → sub-matrix
```

Fancy Indexing & Boolean Indexing

```
a = np.array([10, 20, 30, 40, 50])

# Fancy indexing (select specific indices)
print(a[[0, 2, 4]])          # [10 30 50]

# Boolean indexing (filter by condition)
print(a[a > 25])             # [30 40 50]
print(a[(a > 15) & (a < 45)])# [20 30 40]
```

SECTION B — ARRAY OPERATIONS

10. Arithmetic Operations

NumPy applies arithmetic operations element-wise — no loops needed. This is called vectorization.

```
a = np.array([10, 20, 30, 40])
b = np.array([ 1,  2,  3,  4])

print(a + b)       # [11 22 33 44]   → Addition
print(a - b)       # [ 9 18 27 36]   → Subtraction
print(a * b)       # [ 10  40  90 160] → Multiplication
print(a / b)       # [10. 10. 10. 10.] → Division
print(a // b)      # [10 10 10 10]   → Floor Division
print(a % b)       # [0 0 0 0]       → Modulo (remainder)
print(a ** 2)      # [100 400 900 1600] → Power

# Scalar operations (apply to all elements)
print(a + 100)     # [110 120 130 140]
print(a * 0.5)     # [ 5. 10. 15. 20.]
```

11. Broadcasting Rules in NumPy

Broadcasting allows NumPy to perform operations on arrays of different shapes. Instead of requiring the same shape, NumPy 'stretches' the smaller array to match the larger one.

Broadcasting Rules

- **Rule 1:** If arrays have different number of dimensions, the shape of the smaller one is padded with 1s on the left
- **Rule 2:** Arrays with size 1 along a dimension act as if they are the size of the larger array in that dimension
- **Rule 3:** If sizes differ and neither is 1, NumPy raises an error

```
# Example 1: scalar + array
a = np.array([1, 2, 3])
print(a + 10)              # [11 12 13]  → 10 is broadcast to all

# Example 2: 2D + 1D
A = np.array([[1, 2, 3],   # Shape: (3, 3)
              [4, 5, 6],
              [7, 8, 9]])
b = np.array([10, 20, 30]) # Shape: (3,)
print(A + b)
# b is broadcast across each row:
# [[11 22 33]
#  [14 25 36]
#  [17 28 39]]

# Example 3: column vector + row vector
col = np.array([[1], [2], [3]])   # Shape (3, 1)
row = np.array([10, 20, 30])      # Shape (3,)
print(col + row)
# [[11 21 31]
#  [12 22 32]
#  [13 23 33]]
```

12. Universal Functions (ufunc)

Universal Functions (ufuncs) are NumPy functions that operate element-wise on arrays, returning a new array. They are written in optimized C code and are very fast.

Math ufuncs

```
a = np.array([1, 4, 9, 16, 25])

np.sqrt(a)         # Square root:    [1. 2. 3. 4. 5.]
np.square(a)       # Square:         [1, 16, 81, 256, 625]
np.abs([-3,-1,2])  # Absolute value: [3, 1, 2]
np.exp(a)          # e^x
np.log(a)          # Natural log (ln)
np.log10(a)        # Log base 10
np.log2(a)         # Log base 2
np.power(a, 3)     # a^3 for each element
```

Trigonometric ufuncs

```
angles = np.array([0, np.pi/6, np.pi/4, np.pi/2, np.pi])

np.sin(angles)     # Sine of each angle
np.cos(angles)     # Cosine of each angle
np.tan(angles)     # Tangent
np.degrees(angles) # Convert radians to degrees
np.radians([0, 30, 45, 90, 180])  # Degrees to radians
```

Comparison ufuncs

```
a = np.array([1, 5, 3, 8, 2])
b = np.array([2, 4, 3, 7, 9])

np.greater(a, b)        # [F, T, F, T, F]  equivalent to a > b
np.less(a, b)           # [T, F, F, F, T]
np.equal(a, b)          # [F, F, T, F, F]
np.maximum(a, b)        # [2, 5, 3, 8, 9]  element-wise max
np.minimum(a, b)        # [1, 4, 3, 7, 2]  element-wise min
```

13. Aggregate Functions

Aggregate (or reduction) functions reduce an array to a single value or aggregate along an axis. These are essential for data analysis.

```
data = np.array([[10, 20, 30],
                 [40, 50, 60],
                 [70, 80, 90]])

# Overall aggregates
np.sum(data)           # 450  → sum of all elements
```

```
np.mean(data)          # 50.0 → average of all
np.min(data)           # 10
np.max(data)           # 90
np.std(data)           # Standard deviation
np.var(data)           # Variance

# Axis-based aggregates
np.sum(data, axis=0)   # [120 150 180]  → sum of each column
np.sum(data, axis=1)   # [60, 150, 240] → sum of each row
np.mean(data, axis=0)  # [40. 50. 60.]  → mean of each column
np.max(data, axis=1)   # [30, 60, 90]   → max of each row

# Other useful aggregates
np.cumsum(data)        # Cumulative sum
np.prod(data)          # Product of all elements
np.median(data)        # Median value
np.percentile(data, 75)# 75th percentile
np.argmax(data)        # Index of maximum value
np.argmin(data)        # Index of minimum value
```

SECTION C — ADVANCED

14. Reshaping & Manipulating Arrays

Reshape

Reshape changes the shape of an array without changing its data. The total number of elements must remain the same.

```
a = np.arange(12)            # [ 0  1  2  3  4  5  6  7  8  9 10 11]
print(a.shape)               # (12,)

b = a.reshape(3, 4)          # 3 rows, 4 columns
c = a.reshape(2, 2, 3)       # 3-D: 2 blocks, 2 rows, 3 cols
d = a.reshape(6, -1)         # -1 means 'figure it out' -> (6, 2)

# Flatten: any shape back to 1-D
flat = b.flatten()           # Returns a copy
ravel = b.ravel()            # Returns a view (faster)
```

Stacking Arrays

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.vstack([a, b])     # Vertical stack   -> [[1,2,3],[4,5,6]]
np.hstack([a, b])     # Horizontal stack -> [1,2,3,4,5,6]
np.dstack([a, b])     # Depth stack (3-D)
np.concatenate([a,b], axis=0)  # General-purpose stacking

# Stack along new axis
np.stack([a, b], axis=0)   # [[1,2,3],[4,5,6]]
np.stack([a, b], axis=1)   # [[1,4],[2,5],[3,6]]
```

Splitting Arrays

```
a = np.arange(12).reshape(3, 4)

np.split(a, 3, axis=0)      # Split into 3 equal parts row-wise
np.hsplit(a, 2)             # Split horizontally into 2
np.vsplit(a, 3)             # Split vertically into 3
```

Other Manipulation

```
a = np.array([[1, 2], [3, 4]])

a.T                         # Transpose (swap rows & columns)
np.transpose(a)             # Same as .T
np.flip(a)                  # Reverse all elements
np.flip(a, axis=0)          # Reverse rows
np.rot90(a)                 # Rotate 90 degrees
np.tile(a, (2, 3))          # Repeat array 2x rows, 3x cols
np.repeat(a, 2, axis=0)     # Repeat each row 2 times
```

15. Linear Algebra with NumPy

Linear algebra is the mathematics behind machine learning. NumPy provides a full suite of linear algebra operations through np.linalg.

Matrix Operations

```
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

# Matrix multiplication (most important in ML!)
C = np.dot(A, B)            # Classic way
C = A @ B                   # Python 3.5+ shorthand (preferred)

# Element-wise multiplication (NOT matrix multiply)
D = A * B                   # [[5,12],[21,32]]

# Transpose
print(A.T)                  # [[1,3],[2,4]]
```

np.linalg Functions

```
A = np.array([[1, 2],
              [3, 4]])

np.linalg.det(A)            # Determinant: -2.0
np.linalg.inv(A)            # Inverse matrix
```

```
np.linalg.norm(A)           # Frobenius norm (magnitude)
np.linalg.rank(A)           # Rank of the matrix

# Eigenvalues and Eigenvectors (used in PCA)
vals, vecs = np.linalg.eig(A)

# Solve linear equations: Ax = b
b = np.array([5, 6])
x = np.linalg.solve(A, b)   # Solves for x

# Singular Value Decomposition (SVD — used in ML)
U, S, Vt = np.linalg.svd(A)
```

16. Random Module

The np.random module generates random numbers and is essential for simulations, data sampling, and initialising machine learning models.

Basic Random Functions

```
# Set seed for reproducibility (same 'random' every time)
np.random.seed(42)

np.random.rand(3, 4)            # Uniform [0.0, 1.0), shape (3,4)
np.random.randn(3, 4)           # Standard Normal (mean=0, std=1)
np.random.randint(1, 100, (3,3))# Random ints between 1 and 99
np.random.uniform(5.0, 10.0, 5) # Uniform between 5.0 and 10.0

# Distributions
np.random.normal(170, 10, 1000) # Normal dist: mean=170, std=10
np.random.binomial(10, 0.5, 100)# Binomial: n=10, p=0.5
np.random.poisson(5, 100)       # Poisson: lambda=5
np.random.exponential(2, 100)   # Exponential: scale=2
```

Sampling & Shuffling

```
data = np.array([10, 20, 30, 40, 50, 60])

np.random.shuffle(data)          # Shuffle in-place
np.random.permutation(data)      # Return shuffled copy
np.random.choice(data, 3)        # Pick 3 random elements
np.random.choice(data, 3, replace=False)  # No repetition
```

17. Integration with Other Libraries

NumPy is the bridge between raw data and all major data science tools. Here is how it connects with the most popular libraries.

NumPy + Pandas

```
import pandas as pd
import numpy as np

# Create DataFrame from NumPy array
arr = np.random.randint(50, 100, size=(5, 3))
df  = pd.DataFrame(arr, columns=['Math', 'Science', 'English'])

# Convert DataFrame column back to NumPy array
math_scores = df['Math'].to_numpy()
print(np.mean(math_scores))
```

NumPy + Matplotlib

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)  # 100 points from 0 to 2pi
y = np.sin(x)                         # Sine wave

plt.plot(x, y)
plt.title('Sine Wave')
plt.show()
```

NumPy + Scikit-learn

```
from sklearn.model_selection import train_test_split
import numpy as np

# Create sample data
X = np.random.rand(100, 5)   # 100 samples, 5 features
y = np.random.randint(0, 2, 100)  # Binary labels

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
print(X_train.shape)         # (80, 5)
print(X_test.shape)          # (20, 5)
```

18. Advanced Operations

Sorting

```
a = np.array([30, 10, 50, 20, 40])

np.sort(a)              # [10 20 30 40 50]  → returns sorted copy
a.sort()                # Sorts in-place
np.argsort(a)           # [1, 3, 0, 4, 2]  → indices that would sort it

# Sort 2D along axis
B = np.array([[3,1,2],[6,4,5]])
np.sort(B, axis=1)      # Sort each row
np.sort(B, axis=0)      # Sort each column
```

Set Operations

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([3, 4, 5, 6, 7])

np.union1d(a, b)        # [1 2 3 4 5 6 7]  → all unique values
np.intersect1d(a, b)    # [3 4 5]           → common values
np.setdiff1d(a, b)      # [1 2]             → in a but not in b
np.unique(a)            # Unique elements of a
```

Where & Select

```
a = np.array([10, -5, 30, -20, 50])

# np.where: conditional element selection
result = np.where(a > 0, a, 0)    # Replace negatives with 0
print(result)   # [10  0 30  0 50]

# np.where to get indices
idx = np.where(a < 0)             # (array([1, 3]),)
print(a[idx])                      # [-5 -20]

# np.select: multiple conditions
conditions  = [a < 0, a == 0, a > 0]
choices     = ['negative', 'zero', 'positive']
np.select(conditions, choices, default='unknown')
```

SECTION D — REAL DATA SCIENCE USE CASES

19. Real Data Science Use Cases

19.1 Normalisation

Normalisation scales data to a fixed range (usually 0 to 1). This is a standard preprocessing step before training machine learning models.

```
marks = np.array([55, 72, 38, 90, 61, 45, 83])

# Min-Max Normalisation
normalized = (marks - marks.min()) / (marks.max() - marks.min())
print(normalized)
# [0.327 0.653 0.   1.    0.442 0.135 0.865]

# Z-score Standardisation (mean=0, std=1)
standardized = (marks - marks.mean()) / marks.std()
print(standardized.mean().round(2))   # 0.0
print(standardized.std().round(2))    # 1.0
```

19.2 Handling Missing Values (NaN)

```
data = np.array([12.0, np.nan, 34.0, np.nan, 56.0, 78.0])

np.isnan(data)              # [F, T, F, T, F, F]
np.sum(np.isnan(data))      # 2  → number of missing values

# Statistics ignoring NaN
np.nanmean(data)            # 45.0
np.nanmax(data)             # 78.0
np.nanstd(data)             # Standard deviation (ignores NaN)

# Replace NaN with mean
mean_val = np.nanmean(data)
data[np.isnan(data)] = mean_val
print(data)                 # NaN values replaced with 45.0
```

19.3 Generating Random Data for Simulations

```
np.random.seed(0)

# Simulate student exam scores (normal distribution)
scores = np.random.normal(loc=70, scale=15, size=200)
scores = np.clip(scores, 0, 100)      # Keep between 0 and 100

print(f'Mean   : {np.mean(scores):.2f}')
print(f'Std Dev: {np.std(scores):.2f}')
print(f'Pass % : {np.mean(scores >= 50)*100:.1f}%')
```

19.4 Removing Outliers

```
data = np.array([10, 12, 11, 9, 200, 13, 10, 11, 185])

# Method 1: Clipping
clean = np.clip(data, 0, 50)          # Values above 50 become 50

# Method 2: Z-score filtering
z_scores = np.abs((data - data.mean()) / data.std())
filtered  = data[z_scores < 2]        # Keep only z-score < 2
print(filtered)                        # Outliers removed
```

19.5 Working with Image Data

In computer vision, images are stored as NumPy arrays of shape (height, width, channels).

```
# Simulate a small 4x4 grayscale image (0=black, 255=white)
image = np.random.randint(0, 256, size=(4, 4), dtype=np.uint8)

print(image.shape)           # (4, 4)
print(image.dtype)           # uint8

# Normalise pixel values to 0-1 (standard before feeding to ML)
image_norm = image / 255.0

# Flip image horizontally
flipped = np.flip(image, axis=1)
```

```
# Rotate image 90 degrees
rotated = np.rot90(image)
```

QUICK REFERENCE CARD

20. NumPy Quick Reference

Array Creation

Function	What it Does
`np.array(list)`	Create array from Python list
`np.zeros((r,c))`	Array of all 0s
`np.ones((r,c))`	Array of all 1s
`np.full((r,c), val)`	Array filled with val
`np.eye(n)`	n x n identity matrix
`np.arange(s,e,step)`	Range of values (like Python range)
`np.linspace(s,e,n)`	n evenly spaced values from s to e
`np.random.rand(r,c)`	Random floats 0–1, shape (r,c)
`np.random.randint(l,h,n)`	Random integers between l and h
`np.random.normal(m,s,n)`	Normal distribution: mean=m, std=s

Array Properties & Reshaping

Function / Attribute	What it Does
`a.shape`	Dimensions of the array (rows, cols, ...)
`a.ndim`	Number of dimensions
`a.size`	Total number of elements
`a.dtype`	Data type of elements
`a.reshape(r,c)`	Change shape (same total elements)
`a.flatten()`	Convert to 1-D (copy)
`a.ravel()`	Convert to 1-D (view, faster)
`a.T`	Transpose the array
`a.astype(dtype)`	Convert to a different data type

Math & Statistics

Function	What it Does
`np.sum(a)`	Sum of all elements
`np.mean(a)`	Average value
`np.median(a)`	Median value
`np.std(a)`	Standard deviation
`np.var(a)`	Variance
`np.min(a)`	Minimum value
`np.max(a)`	Maximum value
`np.argmin(a)`	Index of minimum value
`np.argmax(a)`	Index of maximum value
`np.percentile(a,q)`	q-th percentile
`np.cumsum(a)`	Cumulative sum
`np.nanmean(a)`	Mean ignoring NaN
`np.sqrt(a)`	Square root
`np.log(a)`	Natural logarithm
`np.exp(a)`	e raised to the power of each element
`np.abs(a)`	Absolute value

Linear Algebra

Function	What it Does
`A @ B  or  np.dot(A,B)`	Matrix multiplication
`np.linalg.det(A)`	Determinant
`np.linalg.inv(A)`	Inverse matrix
`np.linalg.norm(A)`	Matrix norm (magnitude)
`np.linalg.eig(A)`	Eigenvalues and eigenvectors
`np.linalg.solve(A, b)`	Solve linear system Ax = b
`np.linalg.svd(A)`	Singular Value Decomposition

STUDENT TIPS & PRACTICE EXERCISES

21. Tips for Jeevi Academy Students

Golden Rules to Remember
1. Always import as 'np' — This is the universal standard in every DS project
2. Think in arrays, not loops — NumPy is fastest when loops are avoided
3. Check .shape first — Shape mismatch is the #1 source of NumPy errors
4. Use np.random.seed() — For reproducible experiments and homework
5. Axis 0 = rows, Axis 1 = columns — Keep this in mind for aggregates
6. View vs Copy — Slicing gives a view (shared memory), flatten() gives a copy
7. dtype matters — Use float64 for calculations, int for counts, uint8 for images
8. NumPy -> Pandas -> Scikit-learn — Learn them in this order for best results

Common Mistakes to Avoid

- **Shape mismatch:** Always verify .shape before operations between two arrays
- **Modifying views:** Slices share memory; use .copy() if you need an independent copy
- **Integer division:** np.array([1,2,3]) / 2 gives [0.5, 1.0, 1.5], not [0, 1, 1]
- **Wrong axis:** np.sum(A, axis=0) sums columns, axis=1 sums rows
- **File name conflict:** Never name your Python file 'numpy.py'

22. Practice Exercises

Beginner Level

- Create a 1-D array of even numbers from 2 to 20 using np.arange
- Create a 4x4 matrix of zeros and fill the diagonal with 5
- Find the mean, max, and min of a random array of 10 integers
- Create a 3x3 array and extract the second row and third column
- Reverse a 1-D NumPy array without using a loop

Intermediate Level

- Create a 5x5 matrix of random integers (1-100), then sort each row
- Normalise an array of 8 student marks to the range 0-1
- Create 200 random heights using a normal distribution (mean=165, std=10) and find the percentage of heights above 170
- Given two arrays a and b, find all elements present in a but not in b
- Create a 4x4 matrix and replace all values above 50 with 1 and below 50 with 0 using np.where

Data Science Level

- Load a 2-D dataset, compute mean and std per column, then standardise the entire dataset
- Simulate rolling a die 10,000 times; compute the frequency of each face
- Build a function that removes outliers from a NumPy array using Z-score (threshold = 2)
- Generate a 100x3 dataset using np.random.normal, split 80/20 into train/test sets
- Implement matrix multiplication from scratch and verify the result against np.dot

Happy Learning! — Jeevi Academy

This document is prepared exclusively for Jeevi Academy students.